

G-CAF

GovDigital Composable Architecture Framework

Volume 9

Theories & Foundations

v2.0

Table of Contents

Chapter 1 Theories Library Overview

Generic reusable theories across 6 domains

Chapter 2 PBC Theory

Packaged Business Capability — the composable building block

Chapter 3 Interface Stability Theory

Contract evolution, Postel's Law, and backward compatibility

Chapter 4 Domain-Driven Design Theory

Bounded contexts, aggregates, context maps, and ubiquitous language

Chapter 5 Event-Driven Architecture Theory

Events, producers, consumers, and EDA patterns

Chapter 6 Architecture Principle Theories

API-First, Cloud-Native, Zero Trust, Design for Failure, Loose Coupling

Chapter 7 Governance Theories

Gateway process theory and federated governance

Chapter 8 SRE Theory

SLOs, error budgets, toil elimination, and the four golden signals

Chapter 9 Normalized Systems Compliance

NS theorems and G-CAF implementation

Chapter 10 Extensions Overview

Agent, AI Security, Catalog, Composition, DevSecOps, Vendor extensions

Chapter 1

Theories Library Overview

G-CAF's theories library contains **generic theory modules** that capture reusable architectural and governance concepts independent of G-CAF-specific implementation. These theories can be reused across multiple frameworks, referenced by vendors, and applied to non-government contexts.

Theory vs. Application

Aspect	Theory (Generic)	Application (Specific)
Location	theories/ folder	framework/, gateways/, etc.
Scope	Universal principle	G-CAF implementation
Example	"API-First is a principle where..."	"At Design Gateway, check API-First by..."
Reusability	Cross-framework	G-CAF only
Change Driver	Industry evolution	G-CAF governance decisions

Theory Categories

Category	Theories	Focus
Composability	PBC Theory, Interface Stability Theory	Building blocks and contract evolution
Architecture	API-First, Cloud-Native, Zero Trust, Design for Failure, Loose Coupling	Guiding design principles
Domain Modeling	DDD Theory	Strategic and tactical DDD patterns
Integration	Event-Driven Architecture Theory	Asynchronous communication patterns
Governance	Gateway Process Theory, Federated Governance Theory	Decision-making and multi-entity coordination
Operations	SRE Theory	Reliability engineering and observability

Chapter 2

PBC Theory

A PBC is software that represents a clearly defined business capability, functionally recognizable as a business asset. — Gartner

Core Characteristics

Characteristic	Description
Business-Aligned	Represents a recognizable business capability (e.g., Payment Processing), not a technical function (e.g., Payment Gateway).
Self-Contained	Includes business logic, data storage, APIs, and optional UI — everything needed to deliver its capabilities.
API-First	Exposes capabilities through well-defined APIs before any UI.
Independently Deployable	Developed, released, and scaled by independent teams on their own schedule.
Composable	Can be combined with other PBCs to create composite applications.

PBC vs. Other Patterns

Pattern	Key Difference from PBC
Microservice	PBC is business-aligned; microservice is technically decomposed. A PBC may contain multiple microservices.
SaaS Product	SaaS is a delivery model; PBC is a design pattern. A SaaS product can implement a PBC.
API	API is a communication mechanism; PBC encapsulates a complete capability including logic and data.
Module	Module is a code-level concept; PBC is a business-level concept with independent lifecycle.
Component	Component is generic; PBC is specifically business-capability-aligned with governance requirements.

Industry References

Gartner Composable Business Platform research, MACH Alliance (Microservices, API-First, Cloud-Native, Headless), Nike platform transformation (2,700 systems → 85 platforms), Ally Financial (\$1.2M savings, 70% reduction in rollout effort).

Chapter 3

Interface Stability Theory

Be conservative in what you send, be liberal in what you accept. — Postel's Law

Core Principles

1. **Separate Interface from Implementation** — The external contract is distinct from internal realization. Internal changes do NOT require consumer changes.
2. **Contract-First Design** — Define the interface before implementation. Design → Document → Implement → Iterate on implementation.
3. **Backward Compatibility** — New versions must not break existing consumers.

Compatibility Rules

Change Type	Backward Compatible?
Add optional field	Yes
Add new endpoint	Yes
Remove field	No — Breaking
Change field type	No — Breaking
Rename endpoint	No — Breaking
Change required → optional	Yes
Change optional → required	No — Breaking

Tolerant Reader Pattern

Consumers must implement the Tolerant Reader pattern: ignore unknown fields, handle missing optional fields with defaults, validate only fields the consumer actually uses, and never fail on extra data.

Versioning Strategies

Strategy	Approach	Trade-off
URI Versioning	/api/v1/citizens	Clear but requires URL changes
Header Versioning	Accept: application/vnd.gcaf.v2+json	Clean URLs but less visible
Query Param	?version=2	Simple but pollutes query string
Semantic Versioning	MAJOR.MINOR.PATCH	Standard but needs registry

Chapter 4

Domain-Driven Design Theory

The heart of software is its ability to solve domain-related problems for its user. — Eric Evans

Strategic Design Patterns

Pattern	Description	G-CAF Application
Bounded Context	Semantic boundary where a model applies consistently	Each PBC maps to one or more bounded contexts
Context Map	Documents relationships between contexts	Integration patterns between PBCs
Ubiquitous Language	Shared vocabulary within a context	PBC documentation and API naming
Core/Supporting/Generic Domain	Classification by strategic value	PBC criticality and sourcing decisions

Tactical Design Patterns

Pattern	Description	Example
Aggregate	Cluster of domain objects as a consistency unit	License Aggregate (Applicant, Fee, Document)
Entity	Object with identity that persists over time	Citizen, License, Payment
Value Object	Immutable, defined by attributes not identity	Address, Money, DateRange
Domain Event	Record of something important that happened	LicenseIssued, PaymentCompleted
Repository	Abstraction for data access	LicenseRepository, CitizenRepository
Domain Service	Logic that doesn't belong to a single entity	CrossEntityValidation

Context Mapping Patterns

Pattern	Relationship	G-CAF Usage
Shared Kernel	Two contexts share a subset of the model	Shared reference data between PBCs
Customer-Supplier	Upstream serves downstream's needs	Provider PBC serves Consumer PBC
Conformist	Downstream conforms to upstream's model	Entity PBCs conform to platform standards
Anti-Corruption Layer	Translates between models	Vendor PBC integration via ACL
Published Language	Shared communication model	G-CAF standard contracts and schemas

Chapter 5

Event-Driven Architecture Theory

An event is a significant change in state. When a customer places an order, the order is created — that's an event. — Martin Fowler

Core Concepts

Concept	Description
Event	Immutable record of something that happened (e.g., LicenseIssued)
Event Producer	Component that detects state change and publishes; fire-and-forget
Event Consumer	Component that subscribes and processes; asynchronous
Event Channel	Infrastructure transporting events (Kafka, Service Bus, etc.)

EDA Patterns

Pattern	Description	Use Case
Event Notification	Simple signal that something happened	Trigger downstream processing
Event-Carried State Transfer	Event contains full state snapshot	Reduce inter-service queries
Event Sourcing	Store state as sequence of events	Audit trail, time travel
CQRS	Separate read/write models	Optimized queries, scaling
Saga	Coordinate multi-step transactions	Cross-PBC business processes
Choreography	Services react to events independently	Loosely coupled workflows
Orchestration	Central coordinator directs flow	Complex multi-step processes

CloudEvents Standard

G-CAF mandates CloudEvents specification for all events, providing a common envelope format with standard attributes: specversion, type, source, id, time, data. JSON Schema defines the data payload for each event type.

Chapter 6

Architecture Principle Theories

G-CAF includes 5 detailed architecture theory modules, each capturing industry-standard principles with academic and practical foundations.

Theory	Core Principle	Key Reference	G-CAF Principle
API-First	Design APIs before implementing the product	Open API Initiative	ARC-001
Cloud-Native	Design for cloud from inception, not as an afterthought	Cloud Native Computing Foundation	ARC-002
Zero Trust	Never trust, always verify; assume breach	NIST SP 800-207	ARC-005
Design for Failure	Assume components will fail; design for resilience	Netflix Chaos Engineering	ARC-010
Loose Coupling	Minimize dependencies; communicate via contracts	SOA Principles	ARC-009

Zero Trust — Key Components

Component	Traditional	Zero Trust
Trust Model	Inside firewall = trusted	Every request = verify
Access Scope	Broad role-based, persistent	Just-in-time, just-enough, time-bounded
Review Cycle	Annual	Continuous
Network	Perimeter-based	Microsegmented
Verification	One-time at login	Continuous based on behavior, device, location

Design for Failure — Key Patterns

Pattern	Purpose	Implementation
Circuit Breaker	Prevent cascading failures	Stop calls to failing service; fallback
Retry with Backoff	Handle transient failures	Exponential backoff; jitter
Bulkhead	Isolate failure domains	Separate thread pools/connections
Fallback	Graceful degradation	Cached response; default behavior
Health Check	Detect failures proactively	Liveness and readiness probes

Chapter 7

Governance Theories

Gateway Process Theory

A gate is a checkpoint where a decision is made to commit resources to the next stage. — Robert G. Cooper

Principle	Description
Defined Checkpoints	Work progresses through phases, each ending with formal review
Explicit Criteria	Each gateway has mandatory, conditional, and recommended criteria
Designated Authority	Review Board (collective), Delegate (individual), or Automated
Decision Outcomes	Go (proceed), Kill (stop), Hold (pause), Recycle (rework)
Scalable Governance	Lightweight for low-risk; rigorous for high-risk

Federated Governance Theory

Federated governance enables autonomy at the edges with alignment at the center. — Adapted from Data Mesh

Principle	Description	G-CAF Application
Central Standards, Local Control	Central entity defines standards; local units implement	ARB defines principles; entities implement PBCs
Subsidiarity	Decisions at lowest appropriate level	Strategic = ARB; Tactical = DDA; Operational = Team
Interoperability via Standards	Federated units share standards, not central standard	Standard contracts enable cross-entity PBC reuse
Accountability	Local units accountable for outcomes	Entity architects accountable for PBC compliance
Transparency	Decisions visible and auditable	DDA decision log, PBC Catalog visibility

Chapter 8

SRE Theory

SRE is what happens when you ask a software engineer to design an operations team. — Ben Treynor Sloss, Google

Core Concepts

Concept	Description	G-CAF Application
SLI (Indicator)	Measurable service aspect	Request latency, error rate, throughput
SLO (Objective)	Target value for SLI	99.9% requests < 200ms (per NFR Contract)
SLA (Agreement)	Contract with consequences	Cross-Entity SLA framework
Error Budget	1 - SLO = allowed unreliability	Enables velocity vs. reliability trade-off

Four Golden Signals

Signal	Description	Example Metric
Latency	Time to serve requests	p50, p95, p99 response time
Traffic	Demand on the system	Requests per second
Errors	Rate of failed requests	Error rate percentage
Saturation	System utilization	CPU, memory, disk percentage

Toil Elimination

Toil is manual, repetitive, automatable work that scales linearly with service growth. Target: keep toil below 50% of operational work. G-CAF addresses this through IDP automation, golden paths, and self-service platform capabilities.

Chapter 9

Normalized Systems Compliance

G-CAF is designed with Normalized Systems (NS) principles to ensure **bounded changes produce bounded work**.

NS Theorem Implementation

NS Theorem	G-CAF Implementation	Status
Separation of Concerns	3T+2F architecture; orthogonal classification; PBC Types; Domain Model	Strongly Separated from Governance
Action Version Transparency	Interface Contracts; API versioning; stability tiers; Domain Event Streams	Strong
Data Version Transparency	Schema evolution rules; tolerant reader mandate; versioned schemas	Strong Aggregate boundaries
Separation of State	PBC lifecycle states; Gateway boundaries; clear ownership; Boundaries	Strong Context boundaries
Action Instance Traceability	Audit logging; version tracking in Catalog; Domain Event catalog	Strong
Data Instance Traceability	Audit logging; data lineage (Data Contract); Aggregate-to-Data ownership	Architectural mapping

Key NS Mechanisms in G-CAF

Mechanism	NS Theorem	Description
3T+2F Architecture	SoC	Each tier/frame has distinct responsibility
Orthogonal Classification	SoC	Criticality, Sourcing, Stability vary independently
Interface Contracts	AvT	Stable external APIs separate from internal evolution
Schema Evolution Rules	DvT	Additive-only; no removal; no rename
Tolerant Reader	DvT	Consumers must ignore unknown fields
PBC Lifecycle States	SoS	Clear ownership and gateway boundaries
Change Drivers	SoC	Each module has single reason to change
Version Registry	AvT+DvT	Central tracking of all contract versions

Chapter 10

Extensions Overview

G-CAF is extended through version-specific capability additions that provide specialized governance beyond the core framework.

Extension Catalog

Extension	Version	Purpose	Key Contents
Agent Extension	v1.4+	Agentic AI governance	Agent PBC type, L0–L4 autonomy levels, 8 sub-contracts (Identity, Autonom
AI Security Extension	v2.1	AI/ML security lifecycle	7 contracts (Algorithm, Adversarial, Inference, Privacy, Observability, Traini
Catalog Extension	v1.18	Enterprise catalog governance	Unified metamodel, lifecycle management, quality rules (Gold/Silver/Bronze
Composition Extension	v1.13	UI composition patterns	7 composition contracts (UI, Task, Search, Navigation, Notification, Context
DevSecOps Extension	v2.0	Security in pipelines	DevSecOps standards, pipeline patterns, security testing, tool standards
Vendor Compliance Extension	v2.0	Vendor certification	88 compliance requirements, 4-tier certification, process lifecycle, exit strate

Agent Extension — Autonomy Levels

Level	Description	Human Oversight
L0	Tool-assisted (no autonomy)	Every action requires approval
L1	Supervised execution	Approval required before execution
L2	Bounded autonomy	Exception-based oversight only
L3	High autonomy	Audit-based monitoring
L4	Full autonomy	Policy-based governance only

AI Security Extension — Contract Architecture

The AI Security Extension addresses the full AI lifecycle with 7 specialized contracts under a master contract: Algorithm Security (security-by-design for ML models), Adversarial Defense (robustness against attacks including prompt injection), Inference Security (runtime protection for deployed models), Privacy-Preserving ML (differential privacy, federated learning, homomorphic encryption), AI Observability (drift, bias, and performance monitoring), Training Data Security (data access, integrity, and lifecycle), and AI Incident Response (taxonomy and response procedures for AI-specific incidents).

Volume	Title	Pages
Vol 1	Introduction & Overview	Introductory
Vol 2	Framework Core — Tiers, Frames, Principles, Classifications	Architecture
Vol 3	Contracts & Governance	Compliance
Vol 4	Business Drivers & Strategy	Strategic
Vol 5	Reference Architectures	Technical
Vol 6	Implementation Guide	Operational
Vol 7	Assessments & Compliance	Audit
Vol 8	Templates & Catalogs	Practical
Vol 9	Theories & Foundations (this volume)	Deep-dive

Department of Government Enablement (DGE) | Abu Dhabi
 Enterprise Architecture Office | govdigital.ea@dge.gov.ae